MINI PROJECT FINAL REPORT

Information Security Lab

Student Name(s)	Riddhi Garg Vaibhavi Chaturvedi
Roll No(s)	36 (Reg. No. 240911446) 16 (Reg. No. 240911270)
Programme / Semester / Section	B.Tech / 5th Semester / A
Course	Information Security Lab
Course Code	ICT 3141
Faculty / Guide	Dr. Divya Rao

Academic Year: 2026–2027

NetSentinel: Automated Network Forensics and Evidence-Integrity Platform

Mini Project — Information Security Lab

NetSentinel: Automated Network Forensics Platform

Riddhi Garg
Manipal Institute of Technology
Manipal, India
Roll No. 36 (Reg. 240911446)
riddhi.mitmpl2024@learner.manipal.edu

Vaibhavi Chaturvedi
Manipal Institute of Technology
Manipal, India
Roll No. 16 (Reg. 240911270)
vaibhavi.mitmpl2024@learner.manipal.edu

***Abstract*—Network forensic evidence is only useful in an investigation if it can be shown to be unmodified. This paper presents NetSentinel, a Python and Streamlit platform that analyses packet captures, detects suspicious traffic, binds the resulting evidence to cryptographic protection, and produces a digitally signed report. Rule-based detectors identify port scans, SSH brute-force attempts and look-alike DNS names, and correlate alerts into incidents. Every evidence file (PCAP, image, audio or text) is registered with a SHA-256 digest, an HMAC-SHA256 tag and an RSA-2048-PSS signature over a canonical manifest. An attacker simulator applies twelve modifications to disposable copies, and a defender module recomputes all three checks against the stored reference. On a bundled 70-packet capture NetSentinel raised three alerts grouped into two incidents. Measured benchmarks show SHA-256 hashing a 10 MB file in 7.7 ms, and RSA signing taking 0.394 ms against 0.042 ms for verification. Over 256 single-bit flips the mean avalanche was 128.0 of 256 digest bits (50.0%), and no collisions appeared among 100,000 hashed inputs. All five RSA tamper-resistance checks passed.**

Keywords—network forensics, evidence integrity, SHA-256, RSA-PSS, tamper detection

I. Introduction

A. Background

Network packet captures (PCAP files) are among the most direct records of what occurred during an intrusion, and screenshots, messages and audio recordings are routinely collected alongside them. Courts and incident-response teams need to be able to show that such material has not been altered since collection [1]. Cryptographic hashes, keyed message authentication codes and digital signatures are the standard tools for this purpose [1]–[3].

B. Problem Statement

Investigators often use separate tools for traffic analysis, evidence storage and report preparation. Evidence may be hashed by hand, if at all, and a plain hash can be recomputed by anyone who modifies the file. There is no single workflow that links a detected incident to its evidence, protects that evidence, demonstrates what tampering looks like, and verifies the final report.

C. Research Gap

Intrusion-detection systems such as Snort [5] focus on detection, while forensic guidance such as NIST SP 800-86 [4] describes integrity practice in prose. Few teaching-scale tools combine detection, layered integrity protection, controlled tampering experiments and a signed report in one pipeline, and few report measured costs for each cryptographic layer.

D. Objectives

- Detect port scans, SSH brute force and look-alike DNS names in PCAP files and group alerts into incidents.
- Protect each evidence file with SHA-256, HMAC-SHA256 and RSA-2048-PSS, kept as an immutable reference record.
- Simulate twelve tampering methods on copies and verify that every modification is detected.
- Measure the timing and security behaviour of the primitives and produce a signed PDF report.

E. Paper Organization

Section II reviews related work. Section III describes the architecture, data, algorithms and evaluation metrics. Section IV presents results and discussion. Section V concludes and outlines future work.

II. Related Work

Signature- and rule-based network intrusion detection is well established. Snort performs real-time packet inspection against rule sets [5], and similar tools such as Zeek add event-driven traffic analysis. These are built for live monitoring at scale and do not manage the evidence they flag. NetSentinel uses much simpler fixed-threshold rules, but ties each alert to an incident and to stored evidence.

For forensic readiness, NIST SP 800-86 recommends preserving data integrity with hashes and documenting the chain of custody [4]. NetSentinel follows this by sealing each evidence file at registration and keeping a hash-chained audit trail of every action.

On the cryptographic side, SHA-256 is specified in FIPS 180-4 [1], HMAC in RFC 2104 [2], and RSA signatures with the PSS padding scheme in PKCS #1 v2.2 [3]. The security of PSS was analysed by Bellare and Rogaway [6]. A limitation of the surveyed work is that detection, evidence protection and verification are usually treated separately; this project joins them and measures each step.

III. Methodology

A. System Architecture

NetSentinel is a layered Python application with a Streamlit interface. The pipeline runs: PCAP analysis, threat detection, incident creation, evidence collection, cryptographic protection, attacker simulation, defender verification, security analysis and signed report. Its modules are ingestion (PyShark/TShark parsing into packets and bidirectional flows stored in SQLite through SQLAlchemy), detection (rules and correlation), evidence (integrity engine, manager, tampering, verification and a hash-chained audit trail), crypto (key store, benchmarks and security

experiments) and reports (ReportLab PDF with a detached signature). The interface has five pages: Network Investigation, Evidence Vault, Tampering and Verification, Cryptographic Key Vault, and Comparison and Reports. The end-to-end flow is shown in Fig. 1.

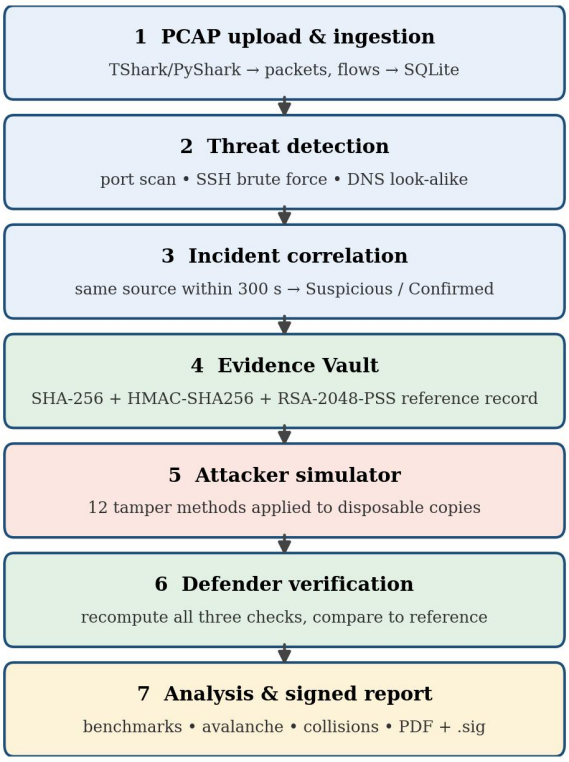


Fig. 1. NetSentinel end-to-end processing pipeline, from PCAP upload to signed report.

B. Dataset

Evaluation uses the bundled synthetic capture attack_demo.pcap: 70 packets over 95 s, of which 62 are TCP, 6 are DNS and 2 are HTTP, forming 41 flows between five distinct IP addresses (46 packets originate from 10.0.0.66). Three synthetic supporting files are used as non-PCAP evidence: a phishing message (text), a screenshot (image) and a voice recording (WAV). All are labelled SIMULATED and are not extracted from the capture. Upload limits are 100 MB (PCAP), 25 MB (audio), 10 MB (image) and 1 MB (text).

C. Preprocessing

Uploaded files are checked for permitted extensions and size, sanitised, and stored permanently as read-only originals. PCAPs are parsed with TShark into per-packet records and grouped into flows, and packet statistics are computed with SQL COUNT and DISTINCT queries rather than loading every row. A structural check reports whether a PCAP or PCAPNG file is parseable or truncated.

D. Algorithm/Model

Detection is deterministic and rule-based. A port scan alert fires when one source probes more than 20 distinct destination ports within 10 s, or sends scan-style TCP probes with abnormal flag combinations to at least three different ports. An SSH brute-force alert needs at least 5 connection attempts to port 22 within 10 s. A DNS look-alike alert fires when a queried name has a string similarity of at least 0.75 to one of six protected brand domains without being an exact match. Alerts that share a source address within a 300 s window form one incident. One firing detector yields a "suspicious pattern"; two or more independent detectors from the same source yield "confirmed malicious activity". This is corroboration, not proof of intent.

Each evidence file receives a reference record that is written once and never regenerated: the SHA-256 digest, an HMAC-SHA256 tag (keyed with a 256-bit secret), and a canonical JSON manifest (evidence ID, incident ID, filename, type, size, digest, time) protected by both an HMAC and an RSA-2048 signature using PSS with MGF1-SHA256 and maximum salt length. Verification recomputes all values from the candidate file and compares them with the stored reference; the manifest is rebuilt from the candidate's digest and size, so any changed byte invalidates the stored signature.

The attacker simulator offers twelve modifications on disposable copies: for images, change one pixel, add a text overlay, or crop; for audio, modify one sample, add noise, or trim; for text, change a character, replace a URL, or modify an IP address; for PCAP, flip one bit, modify a packet field, or truncate the capture. The original and its stored reference are never touched. Reports are signed with a detached RSA-PSS signature (.sig) that can be checked with the exported public key. The verification flow is summarised in Fig. 2.

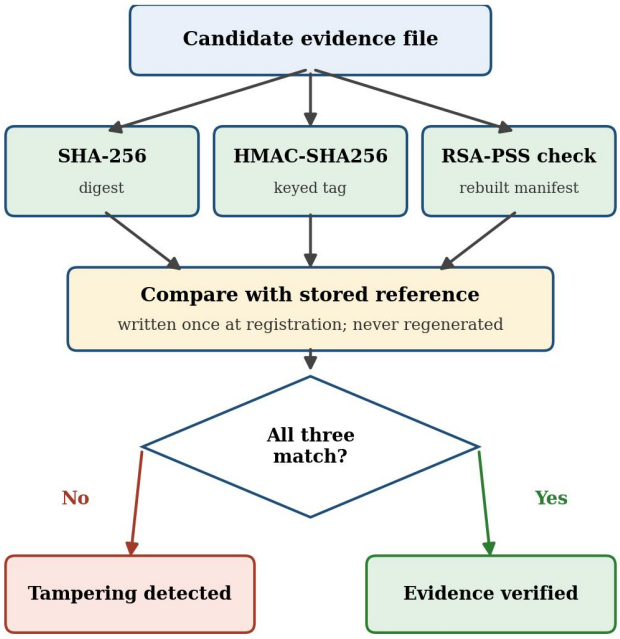


Fig. 2. Defender verification flow: three independent checks are compared with the stored reference record.

E. Experimental Setup

The cryptographic experiments were run in Python with the cryptography library and the project's own benchmark and experiment modules, using a freshly generated RSA-2048 key pair and HMAC secret. Hash and HMAC timing used random inputs of 1 KB, 10 KB, 100 KB, 1 MB and 10 MB, one warm-up call and 15 timed calls using time.perf_counter_ns, reporting the median. RSA timing signed and verified a 512-byte manifest-sized message (30 and 50 timed runs) and timed key generation over three runs. The avalanche test flipped one random input bit of the dashboard's 1,100-byte labelled test message in 256 trials. The collision test hashed 100,000 distinct 32-byte random inputs, plus a 24-bit truncated-digest control on 20,000 inputs.

Results were generated in a Linux sandbox, so absolute times will differ on other hardware.

F. Evaluation Metrics

- Detection: number of packets, alerts and incidents, and the incident classification.
- Performance: median execution time (ms) versus input size, and the sign-to-verify time ratio.
- Avalanche: number and percentage of the 256 digest bits that change after a one-bit input change (ideal 50%).
- Collision resistance: observed collisions among inputs tested.
- Signature security: pass or fail of five negative and positive verification tests.

IV. Results and Discussion

A. Network Detection

On the bundled capture, NetSentinel reported 70 packets in 41 flows and three alerts, grouped into two incidents (Table I). INC001 is a DNS look-alike for paypal.com (paypa1.com, 90% similar) from 10.0.0.20 and is a suspicious pattern with a risk score of 70 (High). INC002 combines a port scan (Medium severity) and an SSH brute force (High severity) from 10.0.0.66 against 10.0.0.10; the scan was recognised from three abnormal-flag SYN probes to different ports, and the brute force from five SSH attempts within 10 s. Two independent detectors agree, so INC002 is classified as confirmed malicious activity with a risk score of 85 (Critical). Because the capture is synthetic and was built to contain these patterns, this shows that the rules work as designed rather than giving a detection rate on real traffic.

TABLE I. Detection Results on attack_demo.pcap

Incident	Source	Alerts	Class	Risk
INC001	10.0.0.20	DNS look-alike	Suspicious	70 High
INC002	10.0.0.66	Port scan; SSH brute force	Confirmed	85 Critical

B. Hash and HMAC Performance

TABLE II. Median Time (ms) vs File Size

File size	SHA-256	HMAC-SHA256
1 KB	0.0013	0.0026
10 KB	0.0078	0.0092
100 KB	0.0735	0.0750
1 MB	0.7669	0.7661
10 MB	7.7210	7.8234

Time grows approximately linearly with size (Figs. 3 and 4): 10 MB takes about 7.7 ms for SHA-256, a throughput of roughly 1.3 GB/s on the test machine. HMAC costs about twice as much as plain SHA-256 for 1 KB inputs (0.0026 ms versus 0.0013 ms) because of the fixed extra hashing of the key blocks, but the difference vanishes for large files (7.82 ms versus 7.72 ms at 10 MB). Evidence files of the sizes accepted by the tool can therefore be fingerprinted and keyed in a few milliseconds.

The plots use a logarithmic time axis, so a straight line indicates that time is proportional to size. Between 1 MB and 10 MB, a tenfold increase in size raised the SHA-256 time from 0.767 ms to 7.721 ms (about 10.1 times) and the HMAC time from 0.766 ms to 7.823 ms (about 10.2 times), which is almost perfectly linear. At the small end the curve is flatter: a 10,240-fold increase in size from 1 KB to 10 MB raised the SHA-256 time by only about 5,900 times, because a fixed per-call cost dominates tiny inputs. In practice, protecting a file adds a delay that is negligible next to the time an investigator needs to upload and review it.

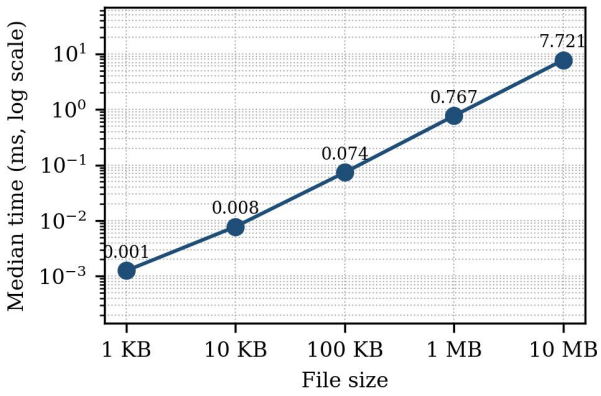


Fig. 3. SHA-256 median time versus file size (log scale).

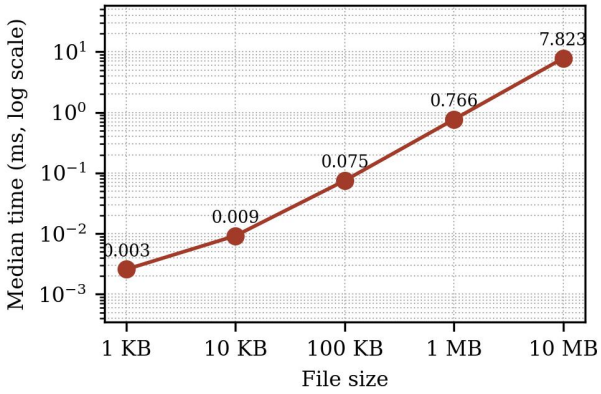


Fig. 4. HMAC-SHA256 median time versus file size (log scale).

C. RSA-2048-PSS Cost

RSA signing took a median of 0.394 ms against 0.042 ms for verification, a ratio of about 9.5:1 (Fig. 5), because signing uses the large private exponent while verification uses the public exponent 65537. Key generation took a median of 17.8 ms and happens once. Signing happens once per evidence file or report, while verification may be repeated by any reviewer, so this asymmetry suits the use case.

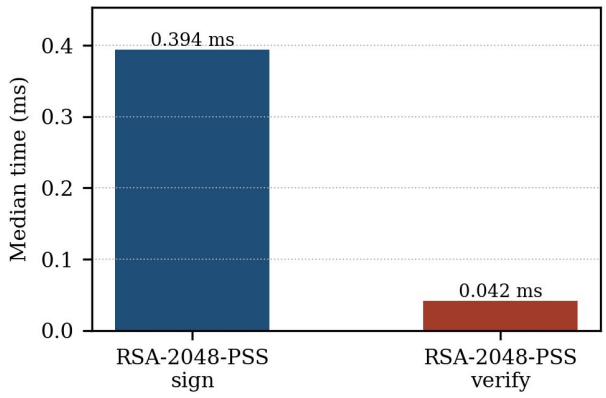


Fig. 5. RSA-2048-PSS median signing and verification time for a 512-byte message.

D. Performance Comparison of the Three Methods

Fig. 6 places the three protection methods on one axis. For files up to 100 KB, SHA-256 and HMAC-SHA256 are cheaper than RSA signing (0.394 ms); hashing only overtakes the signing cost at roughly 500 KB to 1 MB. This is not a like-for-like comparison, since RSA signs a short manifest whose size does not grow with the file, so the cost of the signature layer stays constant while the hash cost grows with evidence size. RSA verification (0.042 ms) is cheaper than hashing a file of 100 KB or more.

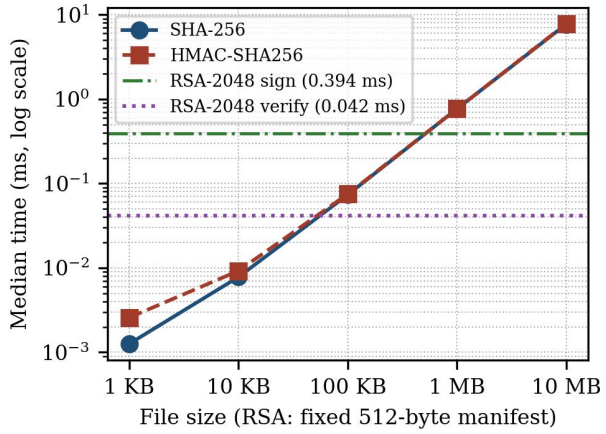


Fig. 6. Performance comparison of SHA-256, HMAC-SHA256 and RSA-2048-PSS (sign and verify).

E. Avalanche, Collisions and Signature Checks

TABLE III. SECURITY EXPERIMENT RESULTS

Experiment	Result
Avalanche, mean of 256 flips	128.0 bits (50.0%)
Avalanche, min / max / std. dev.	103 / 149 / 7.5 bits
SHA-256 collisions in 100,000 inputs	0
24-bit truncated, 20,000 inputs	9 collisions
RSA tamper-resistance checks	5 of 5 passed

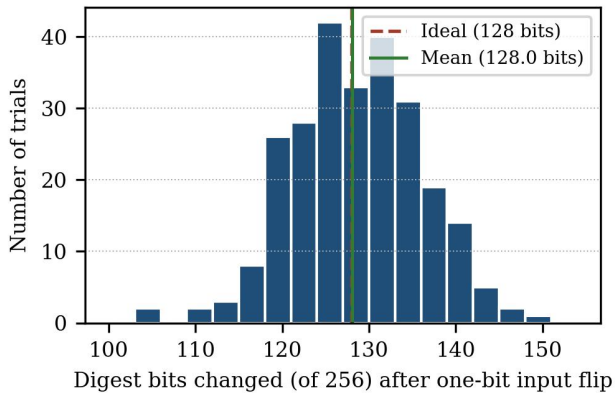


Fig. 7. Avalanche effect: distribution of changed digest bits over 256 single-bit flips.

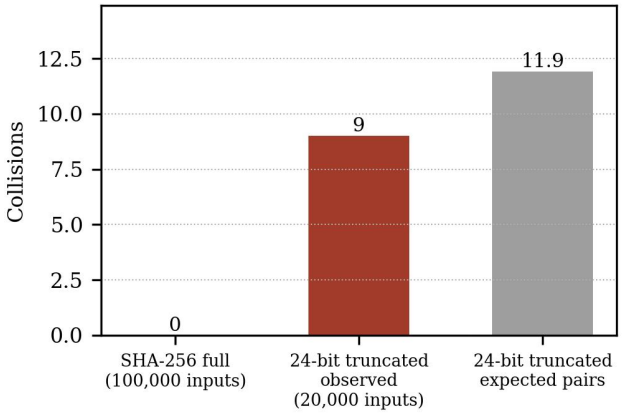


Fig. 8. Collision resistance: full SHA-256 versus a 24-bit truncated digest control.

A single flipped bit changed on average 128.0 of 256 digest bits, matching the ideal 50%, so even a one-pixel edit produces an unrecognisable digest. Zero collisions among 100,000 inputs is expected and is not a proof of collision resistance: finding a SHA-256 collision by generic search needs on the order of 2^128 operations. The 24-bit truncated control, by contrast, collided quickly, which shows why the full 256-bit output matters. The five RSA checks (genuine signature accepted; one-bit file change, wrong public key, one-bit signature change, and an edited manifest field all rejected) behaved as required.

F. Tamper Detection and Limitations

The project's test suite applies each of the twelve tamper methods and then runs defender verification, and all are expected to fail at least the SHA-256 and RSA checks while an untouched original passes. We report this from the test design rather than from a separate large-scale experiment. Several limits remain. HMAC tags sit in the same SQLite file as the records they protect, so their value depends on the secret staying outside an attacker's control. The RSA private key is stored unencrypted for laboratory simplicity and would belong in an HSM or key-management service in production. The audit chain is unsigned, so someone able to rewrite the whole table could forge it; the signed report is the external anchor. Report signatures are detached rather than embedded PDF signatures. The detection rules use fixed thresholds on a synthetic dataset.

V. Conclusion and Future Work

NetSentinel integrates PCAP analysis, rule-based detection, layered cryptographic protection, simulated tampering and signed reporting in one workflow. On the bundled capture it identified three threats in two incidents, and measurements showed that SHA-256, HMAC and RSA verification are cheap enough for routine use, that the avalanche effect is close to the ideal, and that all five signature checks behaved correctly. Contributions are the end-to-end design, the immutable reference record, and the measured evaluation of each primitive.

Future work includes evaluating the detectors on real, labelled traffic and adaptive thresholds, encrypting the private key or moving it to a key-management service, signing the audit chain, embedding PAdES signatures in reports, and adding trusted timestamping and support for more evidence types.

References

[1] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[2] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997.

[3] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," RFC 8017, Nov. 2016.

[4] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to integrating forensic techniques into incident response," NIST SP 800-86, Aug. 2006.

[5] M. Roesch, "Snort: Lightweight intrusion detection for networks," in Proc. 13th USENIX Large Installation Syst. Admin. Conf. (LISA), 1999, pp. 229–238.

[6] M. Bellare and P. Rogaway, "The exact security of digital signatures—How to sign with RSA and Rabin," in Advances in Cryptology—EUROCRYPT '96, LNCS vol. 1070, 1996, pp. 399–416.